GENeva Evolution Code (GENEC): a quick overview

Cyril Georgy & Sylvia Ekström

February 10, 2023

Contents

1 Program **2**

2 First steps **2**
2.1 Getting the program on GitHub . . . 2
2.2 Environment variables . . . 2
2.3 Compilation . . . 3

3 Computation of a model **3**
3.1 Creation of the initial file . . . 3
3.2 Launch of a computation . . . 3

4 Modification of the code **4**
4.1 Branching . . . 4
4.2 Committing . . . 4

5 Parameters card **5**

6 Workflow **10**
6.1 Main program . . . 10
6.2 Resolution of the internal structure equations . . . 10
6.3 Treatment of rotation . . . 11

7 Tools **11**
7.1 Small scripts . . . 11
7.2 Visualisation program: `GENEC_toolBox` . . . 12

8 Annexes **13**
8.1 Main output files format . . . 13
8.2 Modules . . . 16
8.2.1 Modules containing only variables . . . 16
8.2.2 Modules containing variables and subroutines . . . 16

1 Program

GENEC is the stellar evolution program of the Geneva group. It is composed of a main program and 33 associated modules written in fortran 90.

It is maintained on GitHub, in the GESEG organisation (GEneva Stellar Evolution Group).

2 First steps

2.1 Getting the program on GitHub

If you don't have a GitHub account, you need to create one and send your username to sylvia.ekstrom@unige.ch. You will be added as a member of the organisation. Then, you can either download the code from the web interface, or run the following command in a terminal:

```
git clone git@github.com:GESEG/GENEC.git
```

This will install the GENEC directory, which contains three sub-directories: code/, docs/, and tools/. The GENEC/code/ directory will contain:

- all the .f90 modules and programs
- the compilation make_genec file
- the inputs directory containing
  - the opacity files opaAlph_AspCun06.dat,opaAlph_GN93.dat,opaSol_AspCun06.dat,opaSol_GN93.dat and kappa93.dat
  - the ftfp.dat file containing the values of different variables used in the computation of the deformation due to rotation
  - the nuclear reaction rate files vit.datCNE,vit.datCNEO
  - the files netinit.inCNE, netinit.inCNEO describing the advanced network of isotopes
  - the file liste_tables3 listing the paths of the H-b and He-b nuclear reaction rates files
- the taux directory containing
  - all the files of the nuclear reaction rates for H-b and He-b listed in inputs/liste_tables3.

The up-to-date version is the branch develop, so you need to switch to this branch with:

```
git checkout develop
```

2.2 Environment variables

GENEC makes use of environment variables to adapt to the various architecture of its users. Before trying anything with the code, you need to add the following variable in your .bashrc or .bash_profile file in the root directory:

```
export GENEC_INPUT_DIR="/path/to/GENEC/code/"
```

If you installed pgplot you'll need more environment variables (don't forget to modify the paths according to your own architecture):

```
export PGPLOT_DIR="/opt/local/lib"
export PGPLOT_FONT="/opt/local/share/pgplot/grfont.dat"
export PGPLOT_RGB="/opt/local/share/pgplot/rgb.txt"
export PGPLOT_DEV="/XSERVE"
```

2.3 Compilation

Go to the `GENEC/code/` directory. Create the `make` directory, copy the makefile in it:

```
mkdir make
cd make
cp ../make_genec .
```

Edit the `make_genec` file.You need to change the `SRCDIR` to `".."`. If you want to use `pgplot` you need to set `PGPLOT = TRUE`. You can then compile the code with the command:

```
make -f make_genec
```

By default, the executable name is `evosolL2016`, but you can change it in the `make_genec` file in `make`. At compilation time, you can also attribute a new name with

```
make -f make_genec EXEC=XXX
```

Several compilers are proposed in `make_genec`. For the computation of models, we recommend to use `gfortran`, but the others are useful to track bugs when developing some new pieces of code.

You need also to compile the program that creates the initial files:

```
cd /path/to/GENEC/tools/makeini/
make -f make_inifile
```

3 Computation of a model

3.1 Creation of the initial file

In the computation directory, launch the `inifile` program (see Sect. 7). Answer the program's questions about the name of the star, the mass (in $M_\odot$) and metallicity Z, the rotation rate, and if the default settings are ok (solar-scaled abundances of Asplund et al. 2005 with Ne from Cunha et al. 2006, `GENEC` format, pre-calculated initial structure: usually the answer should be yes!).

The program generates a file named `ini_⋆` (with ⋆ the name given in `inifile`), and also the `netdef.in` and `netalu.dat` files.

3.2 Launch of a computation

It is recommended to launch a model with the python script `GENEC_launch.py` that can be found in `GENEC/tools/`.

`GENEC_launch` makes use of environment variables that have to be added in your `.bashrc` or `.bash_profile`:

```
export GENEC_DEFAULT_PROGRAM="/path/to/GENEC/code/make/GENEC_executable"
export GENEC_EMAIL_ADDRESS="your.email@domain.cc"
```

The minimal launching command is:

```
python /path/to/GENEC/tools/GENEC_launch.py starName
```

with `starName` being the name entered when creating the initial file with `inifile`. For the first run based on this initial file, use this command instead:

```
python /path/to/GENEC/tools/GENEC_launch.py starName -i ini_starName
```

By default, the program that is launched is the one stored in the variable `$GENEC_DEFAULT_PROGRAM`. Another executable can be used with the option `-e /path/to/other/executable`. The program can be told to stop at a specific phase (`-p phase_number`, where `phase_number` corresponds to the one written in the `PhysicsParams` list of the parameter card, see Sect. 5) or model number (`-m model_number`).

When the program stops, either because it reached the desired phase/model or because it crashed, it sends an email at the address entered in the variable `$GENEC_EMAIL_ADDRESS`, except if another one is entered with the option `-a other.email@domain.cc`. The email contains the model number and the stop message.

4 Modification of the code

4.1 Branching

Before making any changes, please create your own branch off the development branch using these two commands:

```
git checkout develop
git checkout -b feature/my_feature develop
```

meaning that you create a new branch called "`feature/my_feature`" starting from the "`develop`" branch. Note that `my_feature` should be replaced by an explicit name indicating the topic of the feature you are working on.

You can then work freely on your branch without affecting the others. At any time, you can check which files have been modified by running the command:

```
git status
```

4.2 Committing

When you want to save your modifications to the git repo, use these commands:

```
git add <modified files>
git commit -m <message>
```

where `<message>` is a clear summary of the changes done, and then:

```
git push origin feature/my_feature
```

This pushes the changes you made from your (local) branch to the remote (origin) repository.

When you consider your modifications to be finished and possibly implementable in the regular version of the code, you can log in GitHub and generate a `pull request`.

5 Parameters card

The input parameters card takes the form of a series of named lists, grouping the parameters by affinities. Some parameters have a default value and will not be displayed as long as they are set to this value

`CharacteristicsParams`

contains the name of the model and the model number. The parameters in this list are:

- `STARNAME:` name of the model (free length).
- `MODANF:` `.b` file number (needed to launch a run).
- `NWSEQ:` number of the first model in the time-step series.
- `NZMOD:` number of models computed in a run.
- `END_AT_PHASE:` to stop when a given phase is reached:
  - `2` – end of central H-b;
  - `3` – end of central He-b;
  - `4` – end of central C-b;
  - `5` – end of central Ne-b;
  - `6` – end of central O-b;
- `END_AT_MODEL:` to stop when a given model is reached.

Parameters `end_at_phase=4` and `end_at_model=0` are facultative.

`PhysicsParams`

contains the choices in the included physics and the evolutive phase. The parameters in this list are:

- `IROT:` rotating models if set to `1`.
- `ISOL:` solid rotation if set to `1`.
- `IMAGN:` internal magnetic fields:
  - `0` – no mag. fields
  - `1` – mag. fields included (prescription managed by `n_mag` parameter, see `RotationParams`)
- `IALFLU:` Ne-Na and Mg-Al networks if set to `1`.
- `IANISO:` wind anisotropy if set to `1`.
- `IPOP3:` $Z = 0$ models if set to `1`.
- `IBASNET:` extended nuclear network if set to `1` (future implementation, not yet available).
- `PHASE:` fusion phases:
  - `1` – for central H-b;
  - `2` – for central He-b;
  - `3` – for central C-b;
  - `4` – for central Ne-b;
  - `5` – for central O-b;
  - `6` – for central Si-b.
- `VAR_RATES:` allows to use different reaction rate files if set to `T`. In this case, the file `liste_tables3` must sit in the computation directory and contain the full path toward the reaction rates files to be used.
- `BINTIDE:` tidal interaction in binaries if set to `T`. Uses the following parameters:
  - `BINM2:` the mass of the companion;
  - `PERIODINI:` the initial period of the binary;
  - `CONST_PER:` a switch to keep a constant period if set to `T` (default) or to compute the period evolution otherwise.

Facultative parameters are `imagn`, `ianiso`, `ipop3`, `ibasnet` (default `0`), `var_rates` and `bintide` (default `F`).

`CompositionParams`

contains the chemical characteristics of the model. The parameters in this list are:

`ZINIT`: initial metallicity of the model (needed for the computation of the WR winds).

`ZSOL`: reference solar metallicity (needed for the scaling applied to the mass-loss rates).

`Z`: abundance of the neglected isotopes.

`IKAPPA`: opacity choice:

- `1` – old tabulated opacities (Grevesse & Anders 1991 + Kurucz + Hubner);
- `5` – OPAL tabulated opacities (recommended value);
- `9` – free electron scattering opacities.

`IOPAC`: choice of the opacity table in the case `ikappa=5`:

- `1` – Grevesse & Noels 1993 (solar mixture);
- `2` – Grevesse & Noels 1993 (alpha-enhanced);
- `3` – Asplund+ 2005 + Ne from Cunha+ 2006 (solar mixture);
- `4` – Asplund+ 2005 + Ne from Cunha+ 2006 (alpha-enhanced);
- `5` – Asplund+ 2009 (solar mixture);
- `6` – Asplund+ 2009 (alpha-enhanced).

BEWARE: `zsol` must match the table chosen. For `1` and `2`: `zsol=0.020`; for `3` and `4`: `zsol=0.014`; for `5` and `6`: `zsol=0.0142`.

`zsol=0.014`, `ikappa=5`, and `iopac=3` are facultative.

`RotationParams`

contains the options for the treatment of rotation. The parameters in this list are:

`IDIFF`: computation of the diffusion of Ω and chemicals if set to `1`.

`IADVEC`: advecto-diffusive version f the transport of Ω if set to `1` (usually only during MS).

`ISTATI`: only local conservation of angular momentum if set to `1`.

`ICOEFF`: prescription to be used for the diffusion coefficients:

$D_{\rm shear}$	$D_{\rm h}$		
	Zahn 1992	Maeder 2003	Mathis+ 2004
Maeder 1997	11	12	13
Talon & Zahn 1997	21	22	23
$D_{\rm tot}$ Maeder 2013	31	32	33

`FENERG` fraction of the shear energy used for the mixing;

`RICHAC`: critical Richardson number value;

`IGAMMA`: treatment of the shear according to Maeder & Meynet 1996 if set to `1`.

`FREIN`: magnetic braking if $\neq 0$. The value entered is the one of the magnetic field applied (in [G]).

`K_KAWALER`: `K` parameter in Kawaler 1988 magnetic braking law.

`OMEGA_SATURATION`: $\Omega_{\rm sat}$ in Kawaler magnetic braking law.

`RAPCRILIM`: maximal $\Omega_{\rm crit}$ ratio before the onset of mechanical mass loss. If $\neq 0$, the conservation of angular momentum will be ensured.

`VWANT`: chosen velocity on the ZAMS. It can be entered in three different ways:

- $V_{\rm eq}$ if the value is > 1.0
- $V/V_{\rm crit}$ if the value is $0.0 <$ `vwant` < 1.0
- $\Omega/\Omega_{\rm crit}$ if the value is $-1.0 <$ `vwant` < 0.0

`XFOM`: multiplying factor for surface Ω. Used to reach `vwant` on the ZAMS (automatically set).

`OMEGA`: surface Ω (changes automatically).

`XDIAL,IDIALO,IDIALU`: activate the smoothing of the Ω and μ profiles (used only with the advection).

`ADD_FLUX`: improves the angular-momentum conservation treatment if set to `T`.

`DIFF_ONLY`: applies the tidal mixing only in timesteps where we use diffusion (and not advection) if set to `T`.

`B_INITIAL`: if `>0`, switches on the wind quenching, and evolves the magnetic field strength according to the conservation of magnetic flux.

`ADD_DIFF`: additional viscosity value, used to increase the core-envelope coupling.

`N_MAG`: type of treatment for magnetic fields:

- `1` – Tayler-Spruit dynamo;
- `2` – modified TS (still in exploration);
- `3` – modified TS following Fuller+ 2019 implementation.

`ALPHA_F`: α parameter in Fuller+ 2019. Values higher than `1` lowers the threshold to trigger the instability ($Q_{\rm min}$) and increases the magnetic viscosity (increased transport).

`NSMOOTH`: number of layers used for smoothing the Ω gradient (used by `Mag_diff_general`). Default vaue is `nsmooth=1`, recommended value for Fuller+ 2019 is `nsmooth=5`.

`QMINSMOOTH`: mag. instability always considered if set to `T`.

`istati=0`, `fenerg=1.0`, `richac=1.0`, `gamma=0`, `frein=0.0`, `K_Kawaler=0.0`, `Omega_saturation=14.0`, `vwant=0.0`, `xfom=1.0`, `Add_Flux=T`, `diff_only=F`, `B_initial=0.0`, `add_diff=0.0`, `n_mag=1`, `alpha_F=1.0`, `nsmooth=1`, and `qminsmooth=F` are facultative.

`SurfaceParams`

contains the parameters of the surface (mass loss, fitm, triangle). The parameters in this list are:

`IMLOSS`: choice of the mass-loss prescription:

- `0` – no mass loss;
- `1` – prescription of de Jager+ 1988;
- `2` – manual mass loss (set by `fmlos`);
- `3` – Reimers 1975,1977 formula, η is given by `fmlos`;
- `4` – WR mass loss:
  - WNL: Nugis+ 1998;
  - WNE and WC: Langer 1989 corrected by Schmutz 1997;
- `5` – prescription of Kudritzki & Puls 2000 for $M > 15M_\odot$ and $T_{\rm eff} \geq 3.95$, otherwise de Jager+ 1988;
- `6` – prescription of Vink+ 2001 for $M > 15M_\odot$ and $T_{\rm eff} \geq 3.95$, otherwise de Jager+ 1988;
- `7` – WR mass loss from Nugis & Lamers 2000, with Z dependency according to Eldridge & Vink 2006;
- `8` – WR mass loss from Gräfener & Hammann 2008;
- `9` – prescription of Kudritzki 2002 for $T_{\rm eff} > 4.6$ and very low Z;
- `10` – prescription of van Loon+ 2005 for RSG and AGB;
- `11` – prescription of Vink+ 2001 modified by Markova & Puls 2008, priv. comm. Puls 2010.

`FMLOS`: except for `IMLOSS=2` or `3`, multiplying factor applied to the mass loss. Usually set to `=0.85` during MS, then `=1` (`=3` in case of supra-Eddington layers). If `IMLOSS=2`, `fmlos` sets the wanted mas-loss rates (in $M_\odot$/yr). If `IMLOSS=3`, `fmlos` is the η parameter of Reimers 1975,1977 law: `FMLOS=0.5` for $M_{\rm ini} < 5.5$, and `FMLOS=0.6` for $5.5 \leq M_{\rm ini} < 8.5$.

`RSG_MDOT`: mas-loss recipe used for RSG:

- `0` – default GENEC fit from Crowther 2001;
- `1` – pure de Jager+ 1988;
- `2` – Beasor & Davies 2020.
- `3` – Kee+ 2021
- `4` – van Loon+ 2005

`SUPRAEDDMDOT`: x3 multiplication factor to the wind in case of supra-Eddington layers if set to `T` (default).

`BE_MDOTFRAC`: if not `0`, switches on a progressive mechanical mass loss law $\Delta M_{\rm mech} = f_{\rm Be}\Delta M_{\rm mech}$ with $f_{\rm Be} = \texttt{Be_mdotfrac} + (1 - \texttt{Be_mdotfrac}) \cdot \left(\frac{\Omega/\Omega_{\rm crit} - \texttt{start_mdot}}{\texttt{rapcrilim} - \texttt{start_mdot}}\right)^{64}$. Recommended value is `Be_Mdotfrac=0.1`.

`START_MDOT`: value of $\Omega/\Omega_{\rm crit}$ at which the Be mass loss starts to apply (default 0.80).

`IFITM`: management of the changes of `fitm` during redward evolution (and/or blueward evolution after a RSG phase):

- `0` – manual change;
- `1` – automatic change with a check that `fitm` is still inside the external convective zone;
- `2` – automatic change using a polynomial fit to modify `fitm` (not recommended in case `irot=1`);
- `3` – automatic smooth change, at each time step;
- `4` – automatic smooth change, at the end of each run;
- `5` – automatic smooth change, but only upward (to facilitate the returning to the blue).

`FITM`: mass included in the interior.

`FITMI`: max value of `FITM` to which the star will come back when going back to the blue.

`DELTAL,DELTAT`: triangle size for L and T at the surface;

`NNDR`: management of the behaviour of (L,$T_{\rm eff}$) with respect to the triangle:

- `0` – no verification that L and $T_{\rm eff}$ of the converged model are still in the triangle;
- `1` – verification and search for another triangle if it is not the case;
- `2` – a 20% tolerance on the triangle is applied during verification;

`RSG_Mdot=0`, `SupraEddMdot=T`, `Be_MdotFrac=0.0`, `start_mdot=0.8`, `FITMI=0.98` (no rot) or `=0.999` (rot), and `nndr=1` are facultative.

`ConvectionParams`

contains the parameters of the treatment of convection. The parameters in this list are:

`ILEDOU`: Ledoux criterion for convection if set to `1`.

`IDIFCON`: convection treated as a diffusion if set to `1` (used during O-b et Si-b).

`ELPH`: mixing length for the external convective zone. usually set to `1.6` for models with $M \leq 40 M_\odot$ and to `1.0` for the more massive ones.

`MY`: integration of the envelope on ρ rather than P if set to `1` (used for $M > 40 M_\odot$).

`IOVER`: overshooting parameter:

- `0` – no overshooting;
- `1` – step overshoot given by `dovhp`;
- `2` – variable overshoot from Baraffe+ 2023;

`DOVHP`: value of the α overshooting parameter $d_{\rm over} = \alpha\, H_P$.

`IUNDER`: undershooting taken into account if set to `1`.

`DUNDER`: value of the undershooting parameter.

`iledou=0`, `idifcon=0`, `iunder=0` et `dunder=0.0` are facultative.

`ConvergenceParams`

contains the parameters used for the convergence of the model. The parameters in this list are:

`GKORM`: accepted deviation on the structure variables.

`ALPH`: fraction of the correction applied for the next iteration.

`AGDR`: absolute value of the maximal correction accepted on r, `s`, P et T.

`FAKTOR`: parameter allowing to "compensate" (during the computation only) for layers with $L_r > L_{\rm tot}$, and hence avoid $\mathrm{d}L/\mathrm{d}r < 0$.

`DGRP,DGRL,DGRY,DGRC,DGRO,DGR20:` relative variations (from one layer to the other) accepted on P, L, $X(^4\mathrm{He})$, $X(^{12}\mathrm{C})$ et $X(^{16}\mathrm{O})$.

`NBCHX:` iteration number for the computation of the change in chemical composition.

`NRBAND:` iteration number for the chemicals between model `n` and `n+1`.

`dgr20=0.010`, `nbchx=200` et `nrband=1` are facultative.

`TimeControle`

contains the parameters modifying the time steps. The parameters in this list are:

`XCN:` multiplying factor applied on the time step for the next run.

`ISLOW:` slow version of the program if $\neq 0$ by modification of the ideal nuclear time step `ratxcn`:

- `1` – `ratxcn`/3
- `2` – `ratxcn`/10
- `3` – `ratxcn`/100
- `4` – `ratxcn`/25
- `5` – `ratxcn`/200
- `6` – `ratxcn`/1000

`ICNCST:` constant time step (equivalent to `xcn=1.0`).

`TAUH_FIT:` used to set the maximal timestep in case of critical velocity, as a fraction of the MS lifetime evaluated as:

- `1` – $\log(\tau_{\mathrm{H}}) = A \cdot \log(M) + B$ with
  $A = -2.632$ and $B = 9.827$ for $M \leq 10\,M_\odot$
  $A = -0.715$ and $B = 7.819$ for $M > 10 M_\odot$
- `2` – more recent relation over the full mass domain:
  $\log(\tau_{\mathrm{H}}) = -0.28 \cdot \log(M)^3 + 1.96 \cdot \log(M)^2 - 4.75 \cdot \log(M) + 10.4$

`icncst=0`, and `tauH_fit=1` are facultative.

`VariousSettings`

contains various parameters affecting a run. The parameters in this list are:

`DISPLAY_PLOT:` display of the `pgplot` window if set to `T`.

`IAUTO:` management of the parameters change through the different phases of the evolution:

- `0` – changes have to be done manually (not recommended);
- `1` – obvious changes are done automatically;
- `2` – as above, and also modifications of `gkorm`, `alph`, `faktor`, `deltal`, `deltat` in case the model struggles to converge.

In all cases with `iauto`$\neq$`0`, the changes automatically done by the program are written in the `input_changes.log` file in the computation directory.

`N_SNAP:` sets the frequency at which a snapshot is produced (useful for large `NZMOD`).

`STOP_DEG:` automatically stops a computation when T_{c} becomes degenerate.

`IPRN:` the full structure is printed every `iprn` models. If `iprn` $>$ `nzmod`, only one structure will be printed, the one corresponding to model `nwseq`.

`IA:` final call to `dreckf` before the last call to `henyey` if set to `1`.

`IOUT:` number of layers used for the smoothing of the diffusion gradient at the border of the convective zones.

`ITMIN:` minimal number of iterations in `henyey`;

`XYFILES:` creation of the `.x` et `.y` files used for the post-processing of s-process if set to `T`.

`IDEBUG:` addition of some terminal printings useful for the debugging, with different possible levels. All values > 0 sets `verbose=T`;

`ITESTS`: use of a flag to test some pieces of code;

`VERBOSE`: increases the level of printings on the terminal and the `.l` file.

`n_snap=10`, `stop_deg=T`, `iprn=99`, `ia=1`, `iout=0`, `itmin=5`, `xyfiles=false`, `idebug=0`, `itests=0`, et `verbose=F` are facultative.

6 Workflow

6.1 Main program

The main program is `main.f90`. Very roughly summarised, the workflow is the following:

- reading of a starting structure;
- computation of the deformation due to rotation;
- computation of the mass loss (call to `xloss, xldote, aniso`);
- integration of the atmosphere and the envelope (call to `dreck` and associated sub-routines);
- smoothing of the Ω and μ profiles (call to `grapmui, dlonew`);
- resolution of the structure equations (call to `henyey`);
- verification of the l'acceptability of the solution. In case the solution is rejected, the program starts again with a time step 2x smaller;
- computation of the characteristics of the converged model;
- reintegration of the envelope (call to `dreck`);
- last iteration of the structure equations (call to `henyey`);
- computation of the chemical composition and rotation profile of the next approximative model (call to `chemold, netnew, omenex`);
- printing of the converged model and saving of the approximative model for the next run.

6.2 Resolution of the internal structure equations

The routine is `henyey` in the module `henyey_solver.f90`. Very roughly summarised, the workflow is the following:

- first part, layer by layer treatment:
  - resolution of the equation of state (call to `dichte`);
  - computation of the opacities, of the adiabatic and radiative gradients, and of the nuclear reaction rates (call to `kappa, nabla, energ`);
  - resolution of the structure equations according to the Henyey+ 1964 method, through matrix inversion (call to `gi, zi`);
- second part, global treatment:
  - computation of the variation of Ω and chemicals (call to `vomcon, omenew, netnew`);
  - computation of the advecto-diffusion (call to `diffom, advec`);
- iteration until convergence.

6.3 Treatment of rotation

The treatment of rotation is shared among several sub-routines:

- computation of the deformation of the surface by calling `geocrit, geom, geomat, geomeang`;
- computation of the multiplying factor applied to the mass loss (`alpro6`) in `main`;
- computation of the winds anisotropy in `aniso`;
- computation of the loss of total angular momentum in the time step in `xldote`, and computation of the mechanical mass loss if the model is critically rotating;
- computation of the diffusion coefficients in `coedif`;
- computation of the new abundances after diffusion in `diffbr` and of angular momentum in `diffom`;
- computation of the advective term in `advec` and associated sub-routines.

7 Tools

7.1 Small scripts

Several useful scripts have been developed or adapted for `GENEC`.

The `GENEC/tools/` directory contains the following scripts:

`GENEC_launch.py`: is the launcher for GENEC. You need to set the following environment variables in your `.bashrc` or `.bash_profile`:

- `$GENEC_DEFAULT_PROGRAM` where the path to the default executable is stored;
- `$GENEC_EMAIL_ADDRESS` where your email address is stored for the sending of notifications.

It is called with several input parameters, the only mandatory one being the star's name (P020z14S4, for example). To launch a model from the very beginning, you need to pass the initial file name through option `-i` (`-i ini_P020z14S4.rot`, for example). For informations about the optional parameters, launch it with option `-h`.

`catag.py`: is an utilitary that concatenates all `.g` and `.a` files to generate the evolutive files `.wg` and `.wa`. It takes the star's name as argument. Launched with option `-r`, it generates also a reduced evolutionary file `.dat`.

`MakeInput.py`: is an utilitary that recreates the `.input` file from a previous model. It takes the star's name as argument.

`newstart.py`: is a small script to start a new model from an existing one. It takes the reference directory as argument.

`largest_corr.py`: is a small scripts that looks in the `.l` file given as argument the largest corrections printed just before a negative element. A precise model number can be entered with option `-m num`.

`cleanfiles.bash`: is an utilitary that cleans the computing directory after a model is completed (with a reduction of the total size). It takes the star's name as argument. For informations about the optional parameters, launch it with option `-h`.

`wg_reductor.py`: is an utilitary that reduces the `.wg` file. It takes the star's name as argument. It is used by `catag.py` and `cleanfiles.bash` if they are called with option `-r`.

In addition to the previous scripts, the `GESEG/UtilsEvol/` project contains the following pieces of codes:

`BurningZonesCalc.py`: is an utilitary that runs through all the `.v` files to search for the peaks of H-, He-, and C-burning energies (used in Kippenhahn diagram). It stores the result in a `.burn` file. It is used by `cleanfiles.bash` when it is called with the option `-b`.

`CoresCalc.py`: is an utilitary that computes the various core masses (M_α, $M_{\rm CO}$, $M_{\rm Si}$, $M_{\rm Fe}$) form the `.v` file(s) given as argument. Called without argument, it takes the last `.v` file in the directory.

`PlotData.py`: is an utilitary that uses the python program `GENEC_toolBox` (see Sect. 7.2) to plot the summary of a model from its `.PlotData` binary file.

`UltimateKippenhahn.py`: is an utilitary allowing the creation of Eulerian or Lagrangian Kippenhahn diagrams, with limits of burning zones

7.2 Visualisation program: `GENEC_toolBox`

The python program `GENEC_toolBox` has been developed to process the files generated by `GENEC` and create all kinds of plots. It accepts also SYCLIST tracks, isochrones, and clusters files. It can be retrieved from the GitHub repository `GESEG/GENEC_toolBox`.

It requires `python 2.7` or `3.5`, `numpy >= 1.10`, `scipy >= 0.19`, and `matplotlib >= 2.0`.

When launched for the first time, it creates a file named `.GENEC_toolBox.ini` at the root directory of the user, where it stores the path to the input data it needs and the default path for figures.

A complete manual can be found on GitHub or on our group webpage.

8 Annexes

8.1 Main output files format

.a file (GENEC)

col.	variable	format	quantity	description
1	nm	i6		model number
2	u1	1pe20.13		age
3	u2	9.4	$M/\mathrm{M}_\odot$	
4	u5	e12.6	$X(1)$	)
5	u37	e12.6	$^{3}\mathrm{He}(1)$	)
6	u7	e12.6	$Y(1)$	)
7	u8	e12.6	$^{12}\mathrm{C}(1)$	)
8	u9	e12.6	$^{13}\mathrm{C}(1)$	)
9	u10	e12.6	$^{14}\mathrm{N}(1)$	)
10	u38	e12.6	$^{15}\mathrm{N}(1)$	)
11	u12	e12.6	$^{16}\mathrm{O}(1)$	)
12	u13	e12.6	$^{17}\mathrm{O}(1)$	)
13	u14	e12.6	$^{18}\mathrm{O}(1)$	)
14	u6	e12.6	$^{20}\mathrm{Ne}(1)$	)
15	u11	e12.6	$^{22}\mathrm{Ne}(1)$	)
16	u39	e12.6	$^{24}\mathrm{Mg}(1)$	)
17	u40	e12.6	$^{25}\mathrm{Mg}(1)$	)
18	u41	e12.6	$^{26}\mathrm{Mg}(1)$	) surface abundances
19	u25	e12.6	$^{19}\mathrm{F}(1)$	)
20	u26	e12.6	$^{21}\mathrm{Ne}(1)$	)
21	u30	e12.6	$^{23}\mathrm{Na}(1)$	)
22	u27	e12.6	$^{26}\mathrm{Al}g(1)$	)
23	u28	e12.6	$^{27}\mathrm{Al}(1)$	)
24	u29	e12.6	$^{28}\mathrm{Si}(1)$	)
25	uu(1)	e12.6	$^{28}\mathrm{Si}(1)$	)
26	uu(2)	e12.6	$^{32}\mathrm{S}(1)$	)
27	uu(3)	e12.6	$^{36}\mathrm{Ar}(1)$	)
28	uu(4)	e12.6	$^{40}\mathrm{Ca}(1)$	)
29	uu(5)	e12.6	$^{44}\mathrm{Ti}(1)$	)
30	uu(6)	e12.6	$^{48}\mathrm{Cr}(1)$	)
31	uu(7)	e12.6	$^{52}\mathrm{Fe}(1)$	)
32	uu(8)	e12.6	$^{56}\mathrm{Ni}(1)$	)
33	u15	e12.6	$X(\mathrm{m})$	)
34	u42	e12.6	$^{3}\mathrm{He}(\mathrm{m})$	)
35	u17	e12.6	$Y(\mathrm{m})$	)
36	u18	e12.6	$^{12}\mathrm{C}(\mathrm{m})$	)
37	u19	e12.6	$^{13}\mathrm{C}(\mathrm{m})$	)
38	u20	e12.6	$^{14}\mathrm{N}(\mathrm{m})$	)
39	u43	e12.6	$^{15}\mathrm{N}(\mathrm{m})$	)
40	u22	e12.6	$^{16}\mathrm{O}(\mathrm{m})$	)
41	u23	e12.6	$^{17}\mathrm{O}(\mathrm{m})$	)
42	u24	e12.6	$^{18}\mathrm{O}(\mathrm{m})$	)
43	u16	e12.6	$^{20}\mathrm{Ne}(\mathrm{m})$	)
44	u21	e12.6	$^{22}\mathrm{Ne}(\mathrm{m})$	)
45	u44	e12.6	$^{24}\mathrm{Mg}(\mathrm{m})$	)
46	u45	e12.6	$^{25}\mathrm{Mg}(\mathrm{m})$	)
47	u46	e12.6	$^{26}\mathrm{Mg}(\mathrm{m})$	) centre abundances
48	u31	e12.6	$^{19}\mathrm{F}(\mathrm{m})$	)
49	u32	e12.6	$^{21}\mathrm{Ne}(\mathrm{m})$	)
50	u36	e12.6	$^{23}\mathrm{Na}(\mathrm{m})$	)
51	u33	e12.6	$^{26}\mathrm{Al}g(\mathrm{m})$	)
52	u34	e12.6	$^{27}\mathrm{Al}(\mathrm{m})$	)
53	u35	e12.6	$^{28}\mathrm{Si}(\mathrm{m})$	)
54	uu(1)	e12.6	$^{28}\mathrm{Si}(\mathrm{m})$	)
55	uu(2)	e12.6	$^{32}\mathrm{S}(\mathrm{m})$	)
56	uu(3)	e12.6	$^{36}\mathrm{Ar}(\mathrm{m})$	)
57	uu(4)	e12.6	$^{40}\mathrm{Ca}(\mathrm{m})$	)
58	uu(5)	e12.6	$^{44}\mathrm{Ti}(\mathrm{m})$	)
59	uu(6)	e12.6	$^{48}\mathrm{Cr}(\mathrm{m})$	)
60	uu(7)	e12.6	$^{52}\mathrm{Fe}(\mathrm{m})$	)
61	uu(8)	e12.6	$^{56}\mathrm{Ni}(\mathrm{m})$	)

Evolution `.g` file (GENEC)

n°	variable	format	grandeur	description
1	nm	i6		model number
2	u1	1pe21.15		age
3	u2	f10.6	$M/M_\odot$	
4	xl	f8.6	$\log(L/L_\odot)$	
5	xtt	f8.6	$\log(T_{\rm eff})$	
6	u5	e13.7	$X(1)$	) surface abundances
7	u7	e13.7	$Y(1)$	)
8	b33	1pe13.7	^{3}He(1)	)
9	u8	1pe13.7	^{12}C(1)	)
10	u9	1pe13.7	^{13}C(1)	)
11	u10	1pe13.7	^{14}N(1)	)
12	u12	1pe13.7	^{16}O(1)	)
13	u13	1pe13.7	^{17}O(1)	)
14	u14	1pe13.7	^{18}O(1)	)
15	u6	1pe13.7	^{20}Ne(1)	)
16	u11	1pe13.7	^{22}Ne(1)	)
17	qmnc	f6.4	$M_{\rm cc}$	convective-core mass fraction
18	xte	f8.6	$\log(T_{\rm eff})_{\rm not\ corr.}$	$T_{\rm eff}$ not corrected (*corrwind*)
19	xmdot	f7.3	$\log(\dot{M})$	
20	rhoc	f8.6	$\log(\rho_{\rm c})$	
21	tc	f8.6	$\log(T_{\rm c})$	
22	u15	e14.7	$X(\rm m)$	) central abundances
23	u17	e13.7	$Y(\rm m)$	)
24	b33m	1pe13.7	^{3}He$(\rm m)$	)
25	u18	1pe13.7	^{12}C$(\rm m)$	)
26	u19	1pe13.7	^{13}C$(\rm m)$	)
27	u20	1pe13.7	^{14}N$(\rm m)$	)
28	u22	1pe13.7	^{16}O$(\rm m)$	)
29	u23	1pe13.7	^{17}O$(\rm m)$	)
30	u24	1pe13.7	^{18}O$(\rm m)$	)
31	u16	1pe13.7	^{20}Ne$(\rm m)$	)
32	u21	1pe13.7	^{22}Ne$(\rm m)$	)
33	ybe7m	1pe9.3	^{7}Be$(\rm m)$	)
34	yb8m	1pe9.3	^{8}B$(\rm m)$	)
35	flube7	1pe9.3		neutrino flux from ^{7}Be$(e^-,\nu)^7$Li
36	flub8	1pe9.3		neutrino flux from ^{8}B$(e^+\nu)^8$Be
37	snube7	1pe9.3		as 35 but in SNU units
38	snub8	1pe9.3		as 36 but in SNU units
39	rapcrj	f10.8	$\Omega/\Omega_{\rm crit}$	
40	rot1	1pe9.3	$\Omega(1)$	surface Ω
41	rotm	1pe9.3	$\Omega(\rm m)$	central Ω
42	xobla	1pe9.3	$R_{\rm pol}/R_{\rm eq}$	
43	u27	1pe9.3	^{26}Al$g(1)$	surface abundance
44	u33	1pe9.3	^{26}Al$g(\rm m)$	central abundance
45	alpro6	1pe9.3	$\dot{M}(\Omega)/\dot{M}(0)$	correction of mass loss due to rotation
46	lcnom	i4	$i_{\rm CNO,max}$	layer with max of CNO
47	xmcno	f8.4	$m_{i_{\rm CNO,max}}$	mass of the `lcnom` layer
48	scno	1pe8.2	$\sum$CNO$_{i_{\rm CNO,max}}$	sum of CNO elements in the `lcnom` layer
49	xjspe1	f10.3	$j(3\ M_\odot)$	specific angular momentum within 3 $M_\odot$ [10^{16} cm^2 s^{-1}]
50	xjspe2	f10.3	$j(5\ M_\odot)$	specific angular momentum within 5 $M_\odot$ [10^{16} cm^2 s^{-1}]
51	vcri1m	1pe8.2	$V_{\rm crit,1}$	1st root (before convergence)
52	vcri2m	1pe8.2	$V_{\rm crit,2}$	2nd root
53	vequam	1pe8.2	$V_{\rm eq}$	
54	rapomm	f8.6	$\Omega/\Omega_{\rm crit}$	$\Gamma\Omega$ limit
55	eddesm	f8.6	$\Gamma_{\rm Edd}$	
56	vcrit1	1pe8.2	$V_{\rm crit,1}$	1st root (after convergence)
57	vcrit2	1pe8.2	$V_{\rm crit,2}$	2nd root
58	vequa	1pe8.2	$V_{\rm eq}$	
59	rapom2	f8.6	$\Omega/\Omega_{\rm crit}$	$\Gamma\Omega$ limit
60	eddesc	f8.6	$\Gamma_{\rm Edd}$	
61	dmneed	1pe13.7	$\Delta M_{\rm need}$	mass needed to be lost at the equator (critical velocity)
62	xmdotneed	1pe13.7	$\dot{M}_{\rm need}$	mass loss rate needed to lose $\Delta M_{\rm need}$
63	dlelex	1pe13.7	$\Delta\mathcal{L}_{\rm rad+aniso+mech}$	ang. mom. lost in one timestep (wind + mechanical) [10^{53} g cm^2 s^{-1}]
64	bmomit	1pe13.7	I	total momentum of inertia [10^{57} g cm^2]
65	btot	1pe13.7	$\mathcal{L}_{\rm tot}$	total angular momentum [10^{53} g cm^2 s^{-1}]
66	ekrote	1pe13.7	$E_{\rm rot}$	kinetic energy of rotation [E_{51}]
67	epote	1pe13.7	$E_{\rm pot}$	potential energy [E_{51}]
68	ekine	1pe13.7	$E_{\rm gaz}$	thermal energy of the gas [E_{51}]
69	erade	1pe13.7	$E_{\rm rad}$	radiative energy [E_{51}]
70 - 109	uudrawc	f5.3		limits of convective zones
110	btotatm	1pe16.10	$\mathcal{L}_{\rm tot}$	total angular momentum (star + envelope) [10^{53} g cm^2 s^{-1}]

Structure `.v` file (GENEC)

nº	variable	format	quantity	description
1	j	i4		layer (1 : surface, m : centre)
2	vm	f10.7	M_r/M	
3	x2	f10.7	$\log(P_r)$	
4	x3	f10.7	$\log(T_r)$	
5	x4	f14.11	$\log(r)$	
6	vl	e14.6	$L/L_{\rm tot}$	
7	x	e14.7	X	
8	y	e14.7	Y	
9	xc12	e14.7	^{12}C	
10	xo16	e14.7	^{16}O	
11	eps	1pe11.4	ϵ(H)	energy produced by H-b
12	epsy	1pe11.4	ϵ(He)	energy produced by He-b
13	epsc	1pe11.4	ϵ(C)	energy produced by C-b
14	radm	1pe11.4	$\nabla_{\rm rad}$	(average between `j` and `j+1`)
15	x6	f11.6	$\log(\rho)$	
16	zensi	1pe12.5		convection factor (<0 : rad, >0 : conv.)
17	epsn	1pe11.4	ϵ_ν	energy taken away by neutrinos
18	x10	1pe12.5	$\partial\ln\kappa/\partial\ln P$	
19	x11	1pe12.5	$\partial\ln\kappa/\partial\ln T$	
20	x12	1pe12.5	$\partial\ln E/\partial\ln P$	
21	x13	1pe12.5	$\partial\ln E/\partial\ln T$	
22	x14	1pe12.5	$\partial\ln\rho/\partial\ln P$	
23	x15	1pe12.5	$-\partial\ln\rho/\partial\ln T$	
24	psi	1pe9.2		degeneracy degree
25	epsyy	1pe9.2	$\epsilon(3\alpha)$	
26	epsyc	1pe10.2	$\epsilon(^{12}\mathrm{C}(\alpha,\gamma)^{16}\mathrm{O})$	
27	epsyo	1pe11.2	$\epsilon(^{16}\mathrm{O}(\alpha,\gamma)^{20}\mathrm{Ne})$	
28	eg	1pe12.5	$\epsilon_{\rm grav}$	
29	adim	1pe12.5	$\nabla_{\rm ad}$	(average between `j` and `j+1`)
30	x8	1pe12.5	κ(r)	opacity
31	beta	1pe12.5	$P_{\rm gas}/P_{\rm tot}$	
32	y3	e14.7	^{3}He	
33	xc13	e14.7	^{13}C	
34	xn14	e14.7	^{14}N	
35	xn15	e14.7	^{15}N	
36	xo17	e14.7	^{17}O	
37	xo18	e14.7	^{18}O	
38	xne20	e14.7	^{20}Ne	
39	xne22	e14.7	^{22}Ne	
40	xmg24	e14.7	^{24}Mg	
41	xmg25	e14.7	^{25}Mg	
42	xmg26	e14.7	^{26}Mg	
43	vmyhelio	f9.6	μ	mean molecular weight for layer `j`
44	omegi	1pe12.5	$\Omega(r)$	angular velocity
45	xdmudp	1pe12.5	∇_μ	gradient of chemical composition
46	xricha	1pe12.5	Ri	Richardson number
47	difcon	1pe12.5	$D_{\rm conv}$	convective diffusion coefficient
48	difsch	1pe12.5	$D_{\rm sh}$	shear diffusion coefficient
49	difdef	1pe12.5	$D_{\rm eff}$	diffusion coefficient for meridional circulation and turbulence
50	vmasse	f9.4	M_r	mass included in the shell [$\mathrm{M}_\odot$]
51	dlodlr	e15.8	$\partial\ln\Omega/\partial\ln r$	Ω-gradient
52	xkther	e15.8	K	thermal diffusion coefficient
53	ucicoe	e15.8	$U(r)$	vertical component of the meridional circulation
54	vcicoe	e15.8	$V(r)$	tangential component of the meridional circulation
55	dcirch	e15.8		coefficient of diffusion by meridional circulation (only if `imagn=1`)
56	echpry	e15.8	H_P	pressure scale
57	gravy	e15.8	g	gravity
58	dhori	e15.8	D_h	horizontal-turbulence diffusion coefficient
59	omegp	e15.8	$\Omega(r)$	only local angular momentum conservation
60	vr	e15.8	$r_{\rm prev.}$	radius of the previous model
61	vomegi	e15.8	$\Omega(r)_{\rm prev.}$	Ω of previous model
62	dmago	e15.8	$D_{\rm magn,\Omega}$	magnetic diffusion coefficient for Ω
63	dmagx	e15.8	$D_{\rm magn}$	magnetic diffusion coefficient for the chemicals
64	etask	e15.8	η/K	magnetic over thermal diffusivity
65	bvais2	e15.8	N^2	Brunt-Väisälä frequency
66	bphi	e15.8	B_ϕ	toroïdal magnetic field
67	alven	e15.8	ω_A	Alfven frequency
68	xqmin	e15.8		minimum rotation gradient for the amplification of the magnetic field
69	vmye	f9.6	μ_e	e^- molecular weight
70	xf19	1pe14.7	^{19}F	
71	xne21	1pe14.7	^{21}Ne	
72	xna23	1pe14.7	^{23}Na	
73	xal26g	1pe14.7	^{26}Alg	
74	xal27	1pe14.7	^{27}Al	
75	xsi28	1pe14.7	^{28}Si	
76	xc14	1pe14.7	^{14}C	
77	xf18	1pe14.7	^{18}F	
78	xneut	1pe14.7	n	
79	vxprot	1pe14.7	p	
80	xbid	1pe14.7		
81	abelx(1,8)	1pe14.7	^{28}Si	
82	abelx(2,8)	1pe14.7	^{32}S	
83	abelx(3,8)	1pe14.7	^{36}Ar	
84	abelx(4,8)	1pe14.7	^{40}Ca	
85	abelx(5,8)	1pe14.7	^{44}Ti	
86	abelx(6,8)	1pe14.7	^{48}Cr	
87	abelx(7,8)	1pe14.7	^{52}Fe	
88	abelx(8,8)	1pe14.7	^{56}Ni	
89	btotq	1pe14.7	$\mathcal{L}_{\rm layer}$	total angular momentum of the layer
90	xomegafit	1pe14.7	$\Omega_{\rm fit}$	fit for Ω
91	xmufit	1pe14.7	$\mu_{\rm fit}$	fit for μ
92	1/amu	1pe14.7	μ	used for the fit
93	xoblaj	f8.6	$r_{\rm pol}/r_{\rm eq}$	in the layer

8.2 Modules

Some modules contain only basic variables that are needed in the program and various modules. Others contain a set of variables and subroutines or functions treating the various aspects of the code. The modules are described in the following subsections.

8.2.1 Modules containing only variables

Some variables are used in several modules and need to be declared separately in order to be compiled correctly.

`const:` physical and astrophysical constants;

`evol:` dimension of arrays and path to the `inputs` folder;

`caramodele:` global characteristics of the model (model number, mass, luminosity, age, mass lost, etc ...);

`diffadvmod:` diffusion, advection;

`equadiffmod:` coefficients for the resolution of the differential equations used in `henyey` (structure) and `henadv` (advection);

`rotmod:` rotation variables (Ω, angular momentum, meridional circulation, etc ...);

`strucmod:` structure variables (pressure, temperature, density, gradients, gravity, etc ...).

8.2.2 Modules containing variables and subroutines

These modules are a kind of box treating a specific aspect of the physics of the code.

<u>`abundmod`</u> This module contains the variables for the chemical abundances, reaction rates, neutrinos, energies. It also contains two subroutines:

`abundCheck:` verification that the abundances do not deviate too much from 1;

`maxCNO:` search for the shell with maximal N abundance.

<u>`advection`</u> This module contains the 16 subroutines treating advection:

`advect:` centrale routine for the advection computation, calling all other subroutines and modifying eventually the Ω profile through advection and / or diffusion.

`badv:` determination of the surface conditions for the henyey methode resolving advection;

`confi:` determination of the convective zones configuration (rad-conv-rad, conv-rad, conv-rad-conv, fully rad, fully conv, ...);

`coradv:` computation of the difference between advected Ω and previous Ω;

`ebems:` computation of the components $\bar{\epsilon}/\epsilon_{\rm m}$ (gravitational and nuclear energy) of meridional circulation;

`echtem:` determination of various components of the meridional circulation;

`gbar:` computation of the mean value for g on an isobar;

`giadv:` computation of the matrix elements corresponding to the 4 equations allowing the resolution of the advective part of the angular momentum transport equation;

`gtilgm:` computation of $\tilde{g}/\bar{g}$: fluctuation of g on the isobar with respect to the mean g of the shell;

`henadv:` resolution of the differential equations of advection;

`initia:` initialisation of the meridional circulation according to a simplified expression;

`momcon:` computation of the inertia momentum of the convective zones;

`rechzr:` search for the radiative and convective zones in the model;

`urs1:` computation of U_r, the radial component of the meridional circulation;

`vcirc:` computation of V_r, the horizontal component of the meridional circulation;

`ziadv:` as `giadv`, but for the centre.

<u>`bintidemod`</u> This module contains the 2 subroutines treating the tidal effects:

`dLtidcalc:` computes the tidal forcing on the surface velocity, according to Zahn (1977);

`orbitalevol:` computes the evolution of the orbital separation due to tides.

<u>`chemicals`</u> This module contains the subroutines treating the modifications of chemical abundances due to nuclear reactions, convection and rotational mixing:

`chemold:` homogeneisation of the previous abundances on the new convective zones before the call to `netnew` or `netwki` ;

`chemeps:` homogeneisation of the nuclear reactions in the convective zones before the call to `netnew` (in the case of a call to `netwki`, neutrons and short-lived chemicals are followed, hence no homogeneisation is performed);

`netnew:` computation of the changes in abundances through the nuclear reactions;

`netwki:` idem, but in the case where the NeNa-MgAl network is followed;

`netalu:` called by `netwki`, computation of the matrix elements for the chemicals and H-b reactions;

`netflu:` idem but for He-b reactions;

`chemie:` homogeneisation of the abundances in the convective zones after `netnew` or `netwki`;

`mixe:` called by `chemeps` for the mixing.

<u>`convection`</u> This module contains the routines for the treatment of convection:

`bordn:` search for the borders of the convective zones;

`over1:` computation of the core size extended by the overshooting;

`unders:` computation of the depth of the external convective zone extended by the undershooting;

`fconva:` convection function for $\ell = \alpha \cdot H_\rho$;

`rechzco:` definition of the configuration type:

- `iconra=0:` fully convective
- `iconra=1:` fully radiative
- `iconra=2:` rad—-rad
- `iconra=3:` conv—conv
- `iconra=4:` rad—-conv
- `iconra=5:` conv—rad;

`konvek:` counter for the convective zones in the envelope;

`CZdraw:` determination of the limits of the convective zones for writing in the `.g` file.

`diffusion` This module contains all the routines needed to compute the diffusion of chemicals and angular momentum:

`coedif:` computation of the diffusion coefficients $D_{\rm h}$, $D_{\rm shear}$, $D_{\rm circ}$, $D_{\rm conv}$, and $D_{\rm magn}$;

`couran:` computation of the Courant conditions for the diffusion of the chemicals;

`courom:` idem for the diffusion of Ω;

`diffbr:` resolution of the diffusion equation for the chemicals;

`diffom:` resolution of the diffusion equation for Ω.

`energy` This module contains all the routines needed for computing the nuclear reaction rates and the energy generation:

`energ:` computation of the nuclear reaction rates;

`screen:` computation of the screening factor;

`calcrates:` computation of the nuclear reaction rates and screening factor for the advanced phases;

`interpol:` interpolation in the reaction rates tables;

`netburning:` computation of the abundance modifications due to advanced phases reactions;

`netinit:` initialisation of the advanced phases isotopes network;

`contribreac:` definition of the nuclear reactions considered in the advanced phases;

`readnetZA:` reading of the considered isotopes network;

`readreac:` reading of the considered nuclear reactions;

`inversmat:` inversion of the matrix for the abundance changes due to advanced phases reactions;

`nucal:` computation of the solar neutrino flux;

`nuint:` computation of the integral for the solar neutrino flux;

`enint:` computation of the internal energies of the model:

- gravitational potential energy;
- rotational energy;
- kinetic energy of the gas;
- radiative energy.

`envelope` This module treats the envelope and the atmosphere layers:

`dreckf:` computation of the three α, β, γ of the triangle;

`dreck:` computation of the triangle;

`ggw:` computation of P, T, and r of the envelope and atmosphere;

`atmos:` integration of the atmosphere;

`anfitg:` determination of the conditions in the three first layers of the envelope;

`diff3:` integration of the envelope;

`rsgl:` computation of the right-hand side of the structure equations in the envelope;

`rsgl1:` idem, but in the case of massive stars, with the turbulent pressure taken into account;

`print1:` printing of the structure of the envelope.

EOS This module computes the EOS of the model:

`degen`: computation of the EOS in the degenerate case;

`Compute_sepp`: computation of the variables needed for the EOS (called by `dichte`);

`dichte`: comutations of the EOS.

geomod This module contains the small routines used to interpolate in the table `ftfp.dat`:

`initgeo`: reading of the table `ftfp.dat`;

`geocalc`: interpolation in the table (general application);

`geomat`: from S_ψ determination of r_ψ, $S_{\rm isobare}/(4\,\pi\,r_\psi^2)$, f_T, $g(x_\psi)/\langle g\rangle$, and $\left\langle g^{-1}\right\rangle$;

`geomeang`: from r_ψ determination of $||g(x_\psi)||\cdot\left(\left\langle g^{-1}\right\rangle/g(x_\psi)\right)$;

`geomEdd`: determination of the maximal $\Gamma_{\rm Edd}$ given the value for $\Omega/\Omega_{\rm crit}$;

`geom`: from r_ψ determination of f_P and f_T/f_P

henyey This module contains the routines used for the resolution of the structure equations with the Henyey method:

`printheynyey`: writing of the `.v` file containing the description of the internal structure;

`Calcvmyhelio`: computation of the solar neutrino emission and of μ ;

`gisu`: computation of the elements of the equations $G_{1,2,3,4}$ with the possibility to apply the shift of the point in the shell where the values are computed (Sugimoto 1970, used for the advanced phases);

`gi`: idem in the normal case;

`zi`: computation of the elements of the equations $Z_{1,2,3,4}$;

`henyey`: resolution of the structure equations with the Henyey method.

inputparam This module contains all the variables of the parameters card, as well as the routines that control the changes brought to these parameters:

`Write_param`: writing of the input parameters if they are not set to their default value;

`Write_namelist`: writing of the parameters card;

`Read_namelist`: reading of the parameters card;

`FITM_Change`: computation of the changes in `FITM` during the crossing of the HR diagram;

`IMLOSS_Change`: change of the radiative mass-loss prescription if the model enters a new phase;

`INPUTS_Change`: change of the input parameters depending on the evolutive stage and the current situation of the model.

interpolation This module contains the small routines used for linear or quadratic interpolations, and splines:

`indic,fipoi,fipoi1,fipoi2,quint,qua,quad,quad_gg,flin,spline,splint,getd.`

ionisation This module contains the routines computing the ionisation:

`ionpart`: computation of partial ionisation for H, He, C, O, Ne, Mg;

`saha`: resolution of Saha equation.

`LayersShift` This module contains the routines controlling the layers adjustments and the mesh refinement:

- `FITMshift:` readjustment of the layers after a change in `FITM`;
- `MdotShift:` readjustment of the layers after mass loss;
- `schrit:` addition or suppression of layers according to the variations of given variables from one layer to the next (M, L, r, P, Ω, $X_{\rm H}$, $X_{\rm He}$, $X_{^{12}\rm C}$, $X_{^{16}\rm O}$, $X_{^{20}\rm Ne}$;
- `interx:` interpolation sub-routine used to readjust the values of the variables in the layers that have been modified.

`magmod` This module contains the variables of the magnetic field, and the routine that computes them:

- `Mag_diff:` computation of the variables of the magnetic field, and of the magnetic diffusion coefficients $D_{\rm mag,X}$ et $D_{\rm mag,\Omega}$ (not used anymore);
- `Mag_dif_general:` computation of the magnetic instabilities according to three prescriptions (selected by the `n_mag` parameter):
  - `1:` classical Tayler-Spruit implementation (default value);
  - `2:` modified TS implementation;
  - `3:` Fuller+ 2019 implementation.

`nablas` This module contains the routines computing the nablas.

- `nabla:` computation of $\nabla_{\rm rad}$ and $\nabla_{\rm ad}$, with their derivatives;
- `nabgam:` computation of ∇ in the whole star according to Eq. (6.39) in Maeder (1997);
- `grapmui:` computation of ∇_μ.

`nagmod` This module contains the `NAG` routines used by the program:

- `c02agf:` computation of the roots of a real polynomial equation of `n` degrees;
- `e02acf:` computation of a minimax polynomial fit on a set of points, through Chebyshev polynomials;
- `a02aaf,a02abf,a02acf,c02agt,c02agu,c02agv,c02agw,c02agx,c02agy,c02agz,f06blf:` routines called by the two main ones.

`omega` This module contains the routines treating Ω:

- `VcritCalc:` computation of $V_{\rm crit,1}$, $V_{\rm crit,2}$, $V_{\rm eq}/V_{\rm crit}$, and $\Omega/\Omega_{\rm crit}$;
- `dlonew:` computation of $\mathrm{d}\ln\Omega/\mathrm{d}\ln r$;
- `omenew:` approximate computation of $\Omega(r)$ on the basis of the angular momentum of the previous iteration applied on the new structure;
- `omescale:` computation of the correction to apply to the rotation profile to ensure the conservation of angular momentum after mass loss, and repartition of the correction on a given number of layer under the surface;
- `OmegaReset:` verification that the correction of the angular momentum did not cause $\Omega < 0$ in some layers. if yes, the zones are set to 0 and the correction is recomputed;
- `omconv:` computation of $\Omega(r)$ in convective zones;
- `momevo:` computation of the total angular momentum;
- `mominertenv:` computation of the inertia momentum of the envelope;

`vomcon`: computation of the initial $\Omega(r)$, homogenised in the convective zones, before applying local conservation, diffusion, and advection;

`momspe`: computation of the specific angular momentum;

`omenex`: approximatecomputation of $\Omega(r)$ for the next model;

`om2old`: computation of the inertia momentum (used by `omenex`);

`omesta`: computation of $\Omega(r)$ in case only the local conservation of angular momentum is taken into account (`ISTATI=1, IDIFF=0`).

`opacity` This module contains the routines computing the opacity.

`opacgn93`: determination of the table(s) to be used;

`opac,t6interp,readco,opaltab,fity,fitx,interp,smooth`: sub-routines of the opacity computation recursively called by opacgn93;

`kappa_out`: extrapolation of the opacity if the conditions on T and ρ are out of the tables;

`condTest`: test if conductive opacity has to be computed;

`cond`: computation of the conductive opacity;

`kappa`: front door routine for the computation of the opacities. Calls `opacgn93` or `kappa_out` depending if the conditions are inside the opacity tables or outside.

`PGPlotModule` This module contains all the routines linked to the direct visualisation through `pgplot`. Note that for systems where `pgplot` could not be installed, an alternative version `PGPlotModuleNull` without the `pgplot` commands is present in the source directory and becomes effective if the `PGPLOT` parameter in the makefile is set to `FALSE`.

`InitPGplot`: initialisation of the module and X windows;

`SavePlotData`: saving all the data needed to plot the evolution window, stored in `.PlotData_starName`. Uses `Estimate_Lifetime`, and calls `FindCZ`, `Find_Max_Energy`, and the plotting routines `PlotEvol` and |PlotStruc;

`FindCZ`: search of the convective zones, needed for the Kippenhahn diagram of the evolution window. Calls `Mass_Vector`;

`Mass_Vector`: definition of the mass mesh for the Kippenhahn diagram (y axis);

`Estimate_Lifetime`: estimation of the MS lifetime, based on a fit on Ekström+ 2012, used to set the time tolerance (max timestep between two plotted models if no other variable changed enough for triggering the plotting);

`Find_Max_Energy`: saving the location of the maximal energy generation of the various burnings for the Kippenhahn diagram;

`PlotEvol`: plot of the evolution window;

`PlotStruc`: plot of the structure window;

`EndPGplot`: closing of file and end of module.

`PrintAll` This module contains the routines storing and printing the complete model (interior, envelope, and atmosphere):

`StoreStructure_int,StoreStructure_env,StoreStructure_atm`: storage of the variables of the layers;

`PrintCompleteStructure`: printing of the complete structure.

SmallFunc This module contains all sorts of small routines and general functions:

`girl`: inversion of a matrix;

`CheckProfile`: verification that a curve is smooth, and elimination of the jump if there is one;

`SmoothProfile`: smoothing of a curve by the *locally weighted scatterplot smoothing* technique;

`tridiago`: resolution of the diffusion equation;

`weighed_smoothing`: smoothing of a variable over a defined number of shells;

`threshold_smoothing`: same as above but only if the variable is larger than the threshold;

`primus,drimus`: function $f = \frac{a}{T_9^{e1}} \cdot \exp(-\frac{b}{T_9^{e2}})$ and the derivative, used for the analytical computation of reaction rates in `energ`;

`secun,dsecun`: idem, function $f = \frac{a}{T_9^{e1}} \cdot \exp\left(-\frac{b}{T_9^{e2}} - \left(\frac{T_9}{c}\right)^{e3}\right)$ and derivative;

`tertiu,dterti`: idem, function $f = a + b \cdot T_9^{e1} + c \cdot T_9^{e2} + d \cdot T_9^{e3} + e \cdot T_9^{e4} + f \cdot T_9^{e5}$ and derivative;

`expf10`: ln to $\log_{10}$ function: $f = \exp\left(\frac{x}{\log_{10}(e)}\right)$;

`exphi`: function $f = 1 - \exp(x)$ with limited series expansion if $|x| \leq 0.03$: $f = -\left(\left(\frac{x}{3} + 1\right) \cdot \frac{x}{2} + 1\right) \cdot x$;

`pos`: search for the position of a value in a monotonic table (increasing or decreasing);

`ValInterp`: interpolation function $f = \frac{x_1 \cdot (y_2 - y_3) + x_2 \cdot (y3 - y1)}{y_2 - y_1}$;

`OmInterp`: interpolation function $f = \frac{x_1 \cdot (y_2 - y_3) \cdot z_1 + x_2 \cdot (y_3 - y_1) \cdot z_2}{(y_2 - y_1) \cdot z_3}$.

timestep This module contains the routines controlling the time steps:

`zeit`: computation of the next time step;

`TimestepControle`: adaptation of `xcn` depending on the situation of the model.

winds This module contains the routines linked to mass loss:

`xloss`: computation of the radiatively-driven mass loss;

`xldote`: Computation of the loss of angular momentum;

`dLmagcalc`: computation of the loss of angular momentum due to magnetic braking;

`dLmagLM`: computation of the loss of angular momentum due to magnetic braking in low-mass stars (Kawaler 1988);

`aniso`: computation of the wind anisotropy;

`corrwind`: correction of $T_{\rm eff}$ for the wind thickness;

`Compute_MagCorrection`: computes the reduction of the mass-loss rates due to magnetic quenchiing (Petit+ 2016)

WriteSaveClose This module contains the routines that start and end the computation of a series of time steps:

`write4`: storing of the initial model at the beginning of a model computation;

`read4`: reading of the model stored in `write4`;

`SequenceClosing`: closing of a series of models;

`OpenAll`: opening of the input/output files;

`CloseAll`: closing of the input/output files.